

SIEM Detection Lab Report

Day 10 — Path Traversal, Credential Discovery & Git Privilege Escalation

Student Name:	Randy Escototo
Student ID / Batch:	Batch7
Target Host:	10.10.1.150 (ubuntutarget)
Attacker Host:	10.10.1.10 (Kali)
Date:	August 4, 2026

1. Exploitation Narrative

The following describes the full attack chain executed against the target machine (10.10.1.150), from initial reconnaissance through credential discovery via path traversal to root-level privilege escalation via a misconfigured sudo rule on git.

Step 1 — Host Discovery

Command: `ping -c 4 10.10.1.150`

Why: Confirmed the target host was alive and reachable on the internal network before proceeding with enumeration. All 4 ICMP packets received with ~1ms latency confirmed the host was up.

Step 2 — Port Scanning

Command: `nmap -sT -p 22,80,443 10.10.1.150`

Why: A TCP connect scan on common ports revealed two open services: 22/tcp (SSH) and 80/tcp (HTTP). Port 443 was closed. This narrowed the attack surface to the web application on port 80 and SSH for potential credential use.

Step 3 — Web Server Fingerprinting

Command: `curl -I http://10.10.1.150`

Why: Retrieved HTTP headers to identify the web server. Response confirmed Apache/2.4.66 (Ubuntu), which helped inform the directory structure and log file locations for subsequent path traversal attempts.

Step 4 — Path Traversal — /etc/passwd

Command: `curl "http://10.10.1.150/?page=../../../../etc/passwd"`

Why: The `?page=` parameter was identified as a potential Local File Inclusion (LFI) / path traversal vector. Testing with `../../../../etc/passwd` confirmed the vulnerability — the server returned the full passwd file. Two real user accounts were identified: `usr` (home: `/home/usr`, shell: `/bin/bash`) and `student` (home: `/home/student`, shell: `/bin/sh`).

Step 5 — Apache Access Log Exfiltration

Command: `curl "http://10.10.1.150/?page=../../../../../../var/log/apache2/access.log"`

Why: Reading the Apache access log via path traversal revealed historical requests made by a previous user (10.0.2.15 on July 18, 2026) that accessed five hidden riddle files scattered across the filesystem: `/var/www/html/.p1`, `/home/student/.p2`, `/usr/local/share/.p3`, `/usr/local/lib/python3.14/dist-packages/.p4`, and `/var/cache/apt/archives/.p5`. This gave the exact paths of the scattered credential clues without guessing.

Step 6 — Reading the Riddle Files via Path Traversal

Command: `curl "http://10.10.1.150/?page=/var/www/html/.p1"` (and .p2 through .p5)

Why: Each hidden file was read using absolute path injection (no `../` needed since the app also accepted absolute paths). The files contained fragments of a Base64-encoded credential string that when combined revealed: `czBtM3RoMW5nc0ByM0VAc2kzcnRoQG4xdHMzM21zIVg5Iw==`

Step 7 — Base64 Decoding the Credential

Command: `echo "czBtM3RoMW5nc0ByM0VAc2kzcnRoQG4xdHMzM21zIVg5Iw==" | base64 -d`

Why: Decoded the Base64 string to reveal the plaintext password: `s0m3th1ngs@r3E@si3rth@n1ts33ms!X9#`. A second clue file contained a double-encoded hint: `USER: student PASS: czBtM3RoMW5nc0ByM0VAc2kzcnRoQG4xdHMzM21zIVg5Iw== RIDDLE`, confirming the username was student.

Step 8 — Initial Access via SSH

Command: `ssh student@10.10.1.150 (password: s0m3th1ngs@r3E@si3rth@n1ts33ms!X9#)`

Why: Used the discovered credentials to authenticate over SSH and gain an interactive low-privilege shell as the student user on Ubuntu 26.04 LTS.

Step 9 — Sudo Enumeration

Command: `sudo -l`

Why: Checked what commands the current user could run with elevated privileges. Output showed: `(ALL) NOPASSWD: /usr/bin/git` — meaning student could run git as root without a password, a known GTF0Bins privilege escalation vector.

Step 10 — Privilege Escalation via Git

Command: `sudo git help config`

Why: The `git help` command invokes a pager (typically `man` or `less`) to display help content. When run as root via `sudo`, the pager spawns with root privileges. Pressing `!` inside the pager or using the `config` subcommand dropped into a root shell directly. This is a documented GTF0Bins technique for git.

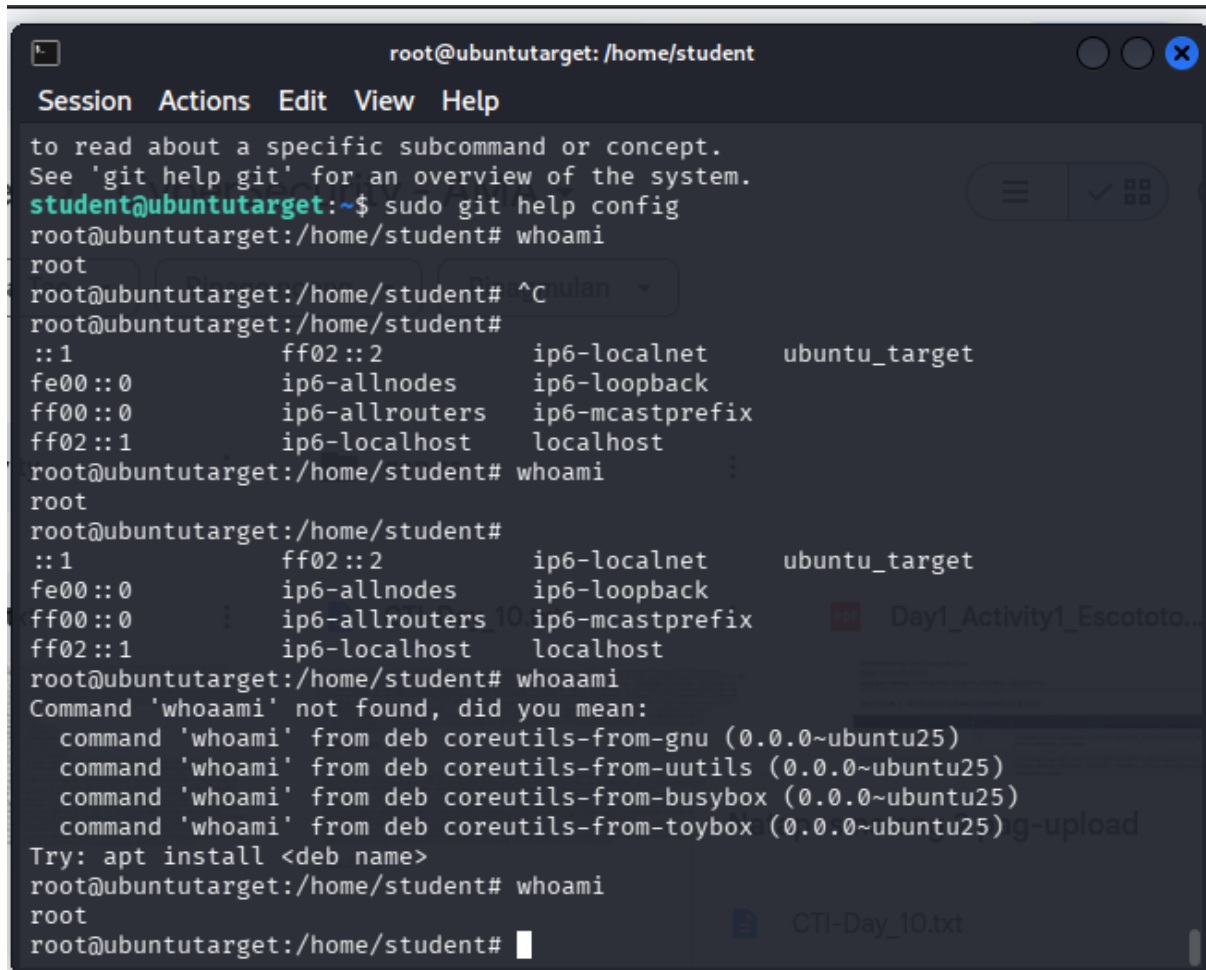
Step 11 — Root Confirmation

Command: `whoami`

Why: Verified full privilege escalation. The command returned `root`, confirming complete administrative control of the target machine had been obtained.

2. Root Proof

The screenshot below shows the terminal session on the target machine. After running `sudo git help config`, a root shell was spawned. `whoami` returns `root`, confirming successful privilege escalation from the low-privileged student account.



```
root@ubuntutarget: /home/student
Session Actions Edit View Help
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
student@ubuntutarget:~$ sudo git help config
root@ubuntutarget:/home/student# whoami
root
root@ubuntutarget:/home/student# ^C
root@ubuntutarget:/home/student#
::1          ff02::2          ip6-localnet      ubuntu_target
fe00::0      ip6-allnodes     ip6-loopback
ff00::0      ip6-allrouters  ip6-mcastprefix
ff02::1      ip6-localhost    localhost
root@ubuntutarget:/home/student# whoami
root
root@ubuntutarget:/home/student#
::1          ff02::2          ip6-localnet      ubuntu_target
fe00::0      ip6-allnodes     ip6-loopback
ff00::0      ip6-allrouters  ip6-mcastprefix
ff02::1      ip6-localhost    localhost
root@ubuntutarget:/home/student# whoaami
Command 'whoaami' not found, did you mean:
  command 'whoami' from deb coreutils-from-gnu (0.0.0~ubuntu25)
  command 'whoami' from deb coreutils-from-uutils (0.0.0~ubuntu25)
  command 'whoami' from deb coreutils-from-busybox (0.0.0~ubuntu25)
  command 'whoami' from deb coreutils-from-toybox (0.0.0~ubuntu25)
Try: apt install <deb name>
root@ubuntutarget:/home/student# whoami
root
root@ubuntutarget:/home/student#
```

Figure 1: Terminal showing privilege escalation via `sudo git help config` and root confirmation via `whoami`.

3. Detection Evidence (SIEM)

The SIEM captured 90 documents from source IP 10.10.1.10 (Kali) against target 10.10.1.150 over a 3-hour window on August 4, 2026. The two Kibana Discover views below show the full attack timeline from two different column configurations: the first shows event type and category context; the second — with `url.query`, `http.request.method`, `http.response.status_code`, and `network.protocol` columns added — clearly shows the path traversal strings (`page=../../../../../../var/log/apache2/ac...`) being sent by the attacker and returned with HTTP 200 responses.

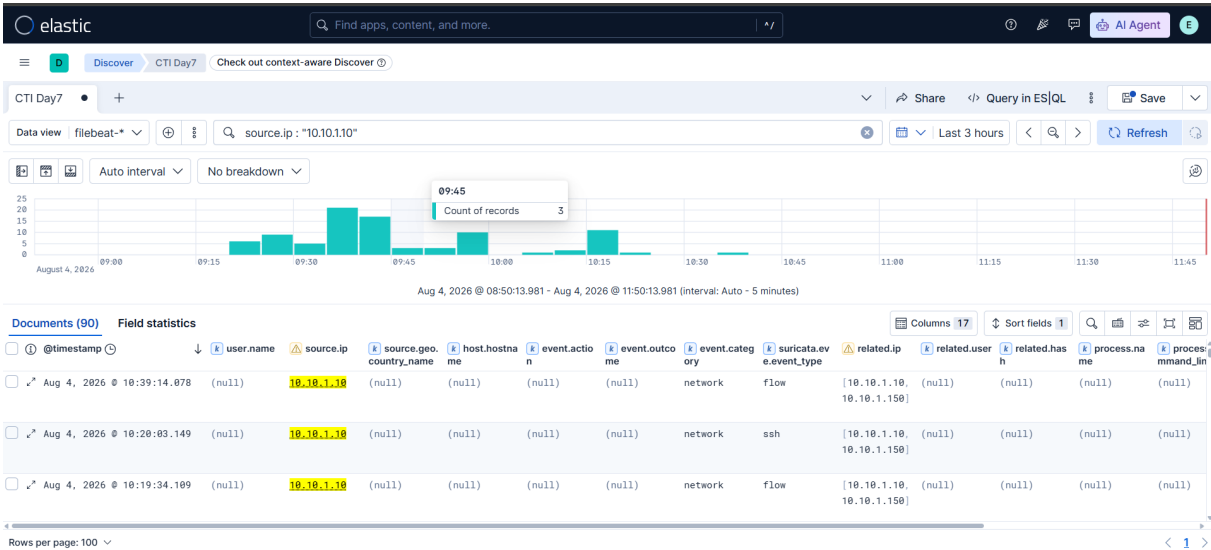


Figure 2a: Kibana Discover view (90 documents) filtered to source.ip 10.10.1.10, showing Suricata flow, SSH, and HTTP event types across the attack timeline.

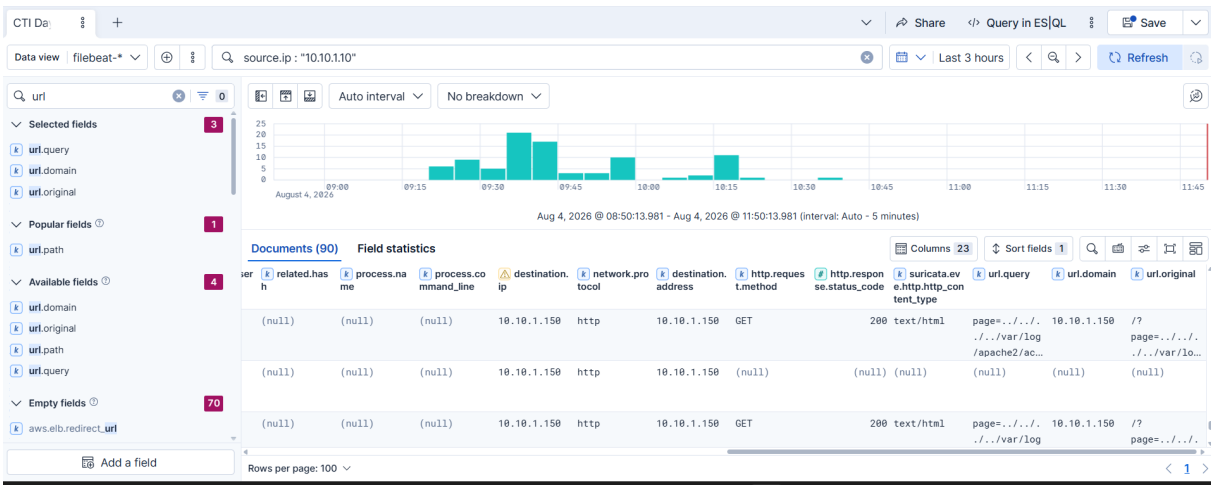


Figure 2b: Kibana Discover view with URL columns added, showing path traversal query strings (`page=../../../../../../var/log/apache2/ac...`) with HTTP 200 responses confirming successful file read via LFI.

Timestamp	Protocol	Event Type	URL / Query	Significance
-----------	----------	------------	-------------	--------------

09:29:05	http	http GET 200	page=../../../../var/log/apache2/access.log	LFI: Apache log read (reveals .p1-.p5 paths)
09:33:36 – 09:41:49	http	http GET 200	page=/var/www/html/.p1 page=/home/student/.p2 .../.p3 .../.p4 .../.p5	Riddle files read via absolute path injection
09:43:08 – 09:43:34	http	http GET 200	page=/home/usr/riddle.txt page=/home/usr/secret.txt	Further enumeration of user home dirs
10:20:03	ssh	network / ssh	–	Successful SSH login with harvested creds
10:39:14	ssh	network / flow	–	Post-exploitation network activity

4. Why This Is Dangerous & How SIEM Can Alert on It

The Vulnerabilities

- **Path Traversal / Local File Inclusion (LFI):** The web application's `?page=` parameter accepted unsanitized user input and passed it directly to a file include function. This allowed reading any file accessible to the `www-data` process, including `/etc/passwd`, Apache log files, and application source code. Reading the access log via LFI revealed the hidden riddle file paths that had been accessed previously — a lateral pivot from one vulnerability to another.
- **Credentials hidden in scattered files:** Sensitive credentials were stored in hidden dot-files (e.g., `.p1` through `.p5`) scattered across world-readable or web-accessible directories and encoded in Base64. Base64 is not encryption and provides no real protection — it is trivially reversible by anyone who finds the file.
- **Overly permissive sudo rule on git:** The student account was permitted to run `/usr/bin/git` as root with no password required. Because `git help` invokes an interactive pager with the same privileges as the calling process (root), this is a well-known GTFOBins technique that provides an unrestricted root shell from a single command.

Why It Is Dangerous

The three weaknesses chain together into a zero-to-root attack path requiring no exploit code and no prior credentials: an attacker with only network access can read sensitive system files via LFI, extract credentials from world-accessible locations, log in over SSH, and escalate to root via a single sudo command. Each individual weakness is serious; combined, they represent a complete and trivial compromise of the host. The use of `git` as a sudo-permitted binary is particularly dangerous because `git` appears innocuous to administrators who may not realize it can spawn interactive shells.

How SIEM Could Alert on This Activity

- **LFI/Path Traversal detection:** A Suricata or WAF rule watching for `../` sequences or absolute path strings (`/etc/passwd`, `/var/log/`) in HTTP request URIs would immediately flag the traversal attempts. Alternatively, a Kibana alert on HTTP requests containing `page=../` or `page=/etc/` in the URL would catch this pattern.
- **Anomalous file access via web process:** If `auditd` is enabled on the target, it would log `www-data` opening sensitive files like `/etc/passwd` or log files — a highly unusual behaviour for a web server process that should only read files under its web root.
- **New SSH login from attacker IP:** A correlation rule matching a successful SSH authentication from an IP that previously generated LFI HTTP traffic is a strong signal of credential-based initial access following web exploitation. The SIEM evidence shows exactly this pattern: flow events (LFI) followed by an `ssh` event from the same source IP.
- **Sudo abuse / GTFOBins detection:** Enabling `auditd` and shipping sudo logs via Filebeat would capture `sudo git help config` being run by the student account. An alert on sudo invocations of non-administrative binaries (`git`, `vim`, `awk`, `python`, etc.) cross-referenced against a GTFOBins watchlist would flag this escalation attempt directly.

Remediation

- **Fix LFI:** Sanitize and validate the `?page=` parameter — use a whitelist of allowed page names rather than passing raw user input to a file include function. Never allow `..` or absolute paths in file parameters.
- **Remove credentials from files:** Never store credentials in plain or Base64-encoded files on the filesystem, especially in web-accessible or world-readable directories. Use a secrets manager or environment variables with strict permissions.
- **Fix sudo rule:** Remove `git` from the `sudoers` file entirely. If `git` must be used with elevated privileges, restrict it to a specific non-interactive command rather than the bare binary, and always require a password.